UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Unidad Académica de Algoritmos y Programación
CAO403: Programación II

Autores:
Álvaro Espinoza
Miguelangel Vasquez
Gustavo Rivas

Guía de Ejercicios
Estructuras Jerárquicas: Árboles

1) Se define el isomorfismo de árboles como una biyección de los nodos de un grafo sobre otro, de modo que se preserva la adyacencia de los vértices. Más formalmente, el isomorfismo de dos árboles A y B es una biyección f entre los conjuntos de sus nodos f: $N(A) \rightarrow N(B)$ que preserva la relación de adyacencia. Es decir, cualquier par de nodos u y v de A son adyacentes si y sólo si lo son sus imágenes, $f(u)$ y $f(v)$ en B. Esta definición matemática del problema es muy elegante, formal y puede causar confusión y temor en el estudiante que está presentando esta evaluación. Sin embargo, solo fue colocada para ocupar espacio en la hoja. Realmente dos árboles son isomorfos si y sólo si su arquitectura es igual sin importar el contenido de sus nodos. Ejemplo:

```
Árbol 1     Árbol 2
   1           7
  / \         / \
 2   3       6   8
```

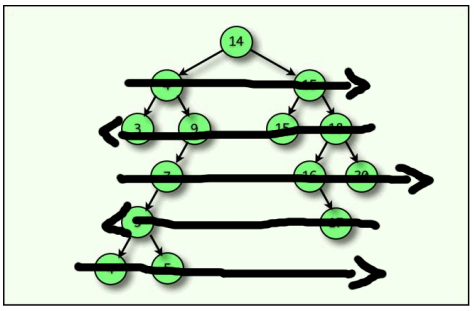
Implemente el método:

func ArbolBin<int>::esIsomorfo(ArbolBin<int>: b): bool

que retorna verdadero si ambos árboles son isomorfos. UTILICE SOLO APUNTADORES en su solución y esta debe ser a lo sumo ***O(n)***.

---

2) Recorrido en Zig-zag: dado un árbol binario, imprima los datos del árbol binario en orden zigzag: recorrer los niveles de un árbol binario en un orden zigzag (nivel 1 de izquierda a derecha, nivel 2 de derecha a izquierda, y así sucesivamente).



proc ArbolBin<elemento>::imprimirZigZag()

Debe ser a lo sumo ***O(n)***. UTILICE APUNTADORES EN SU SOLUCIÓN.

3) Suponemos la clase **"Personaje"** con dos métodos: **"esDragon()"** y **"esPrincesa()"** que permiten determinar si un personaje es un dragón o es una princesa, respectivamente.
   Suponemos también que ningún personaje es dragón y princesa a la vez, pero que un personaje puede no ser ninguna de las dos cosas.
   Dado un árbol n-ario A cuyos elementos son del tipo "Personaje", se denominan nodos accesibles aquellos nodos tales que a lo largo del camino desde la raíz hasta el nodo (ambos inclusive) no se encuentra ningún dragón.

Crear una función:

func princesaMasCercana(ArbolN<Personaje>: arbol): string

De a lo sumo ***O(n)*** que encuentre el nombre de una princesa lo más cerca posible de la raíz de A. Suponiendo que algún nodo accesible de la raíz contiene una princesa. NO UTILICE APUNTADORES.

---

4) Se dice que un árbol binario es AVL si es Binario de Búsqueda y se cumple que:
   $A =< e1, A1, A2 >$
   ● $| Altura(A1) - Altura(A2) | \leq 1$
   ● $A1$ y $A2$ también son árboles AVL
   Por definición el árbol nulo es AVL.

Se pide que cree el método:

func ArbolBin<int>::esAVL(): bool

Que determine en tiempo de complejidad a lo sumo $O(n)$ si el árbol binario instanciado es AVL. DEBE USAR APUNTADORES EN SU SOLUCIÓN. Asuma que existe la función
func abs(entero: a):entero

---

5) Dado un árbol N-ario de entero y 2 números n y m pertenecientes al árbol. Encontrar el camino entre ellos 2.

func camino(ArbolN<elemento>: A): Lista<elemento>

Complejidad máxima ***O(n)***. NO UTILICE APUNTADORES EN SU SOLUCIÓN.

---

6) Dados dos nodos de un árbol binario, encontrar el ancestro común más bajo.

func ArbolBin<elemento>::getLowerCommonAncestor(): elemento

Complejidad de a lo sumo ***O(n)***. UTILICE APUNTADORES EN SU SOLUCIÓN.

---

7) Diámetro de un árbol binario: Calcula el diámetro (la longitud del camino más largo entre dos nodos cualesquiera) de un árbol binario.

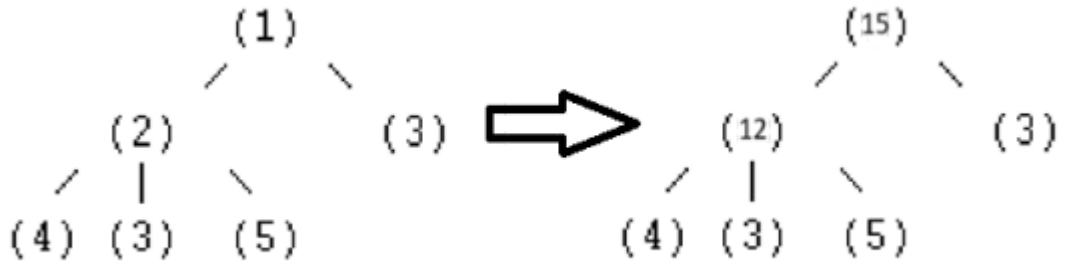
func ArbolN<elemento>::getDiametro(): int

Complejidad máxima de ***O(n)***. UTILICE APUNTADORES EN SU SOLUCIÓN.

8) Extienda la clase ArbolN. Cree el método:

func ArbolN<int>::getArbolSuma(): ArbolN<int>

Que retorne el 'árbol suma' del árbol dado. Por ejemplo: cada nodo contiene la suma de los valores de todos los nodos en su subárbol



Su solución debe tener una complejidad en tiempo de a lo sumo ***O(n)***. DEBE UTILIZAR SOLO APUNTADORES EN SU SOLUCIÓN. Sólo se permiten los constructores.

---

9) Sea A un árbol n-ario de entero, y N un número perteneciente al árbol.

Cree la función:

func ArbolN<int>::getPrimos(int: n): Lista<int>

Que devuelva todos los primos de n (aquellos nodos que son hijos de los tíos de n). UTILICE APUNTADORES EN SU SOLUCIÓN. ***O(n)***

---

10) Dada una lista. Ordenarla usando árboles (SIN UTILIZAR APUNTADORES). Asuma para este ejercicio que todas las clases de árboles tienen un método Preorden, Inorden y Postorden que devuelven una lista con el recorrido.

Debe crear la función:

func ordenar(Lista<int>: l, cadena: tipo): Lista<int>

En donde el parámetro "tipo" puede tomar cualquiera de los siguientes valores ["ascendente", "descendente"].

La solución debe ser a lo sumo **O(nlog(n))**.

---

11) Encuentra el camino en un árbol binario que tiene la suma máxima de valores de nodos.

func ArbolBin<int>::sumaMaxima(): Lista<int>

Complejidad a lo sumo ***O(n)***. UTILICE APUNTADORES EN SU SOLUCIÓN.

12) Se define la “frontera” de un árbol n-ario como aquellos nodos en la parte externa del árbol (ver sombreado):

Se requiere que implemente la función:

func ArbolN<int>::sumaInterna(): int

Que sume todos los nodos internos, o sea, aquellos que NO pertenecen a la frontera. La solución debe ser a lo sumo ***O(n)***. UTILICE APUNTADORES EN SU SOLUCIÓN.

---

13) En la creación de compiladores, en los algoritmos de tu calculadora y en otras áreas de la ingeniería y computación se utilizan una variación de los árboles binarios, conocido como árbol de sintaxis. Son utilizados para llevar a cabo operaciones aritméticas combinadas donde las operaciones tienen jerarquía. En el caso de la aritmética que manejamos comúnmente, en ellos las hojas del árbol serían los operandos, y el resto de nodos interiores vendrían a ser operadores (+, -, *, /).

Dado una lista de caracteres donde está inscrita una operación combinada, y considerando la jerarquía entre operaciones (* y / son de mayor prioridad que + y -); extienda la clase ArbolBin con un constructor a partir de una lista, que genere un árbol binario de sintaxis.

ArbolBin<char>::construir(Lista<char>: op)

La complejidad a lo sumo debe ser ***O(n)***. USE SOLO APUNTADORES EN SU SOLUCIÓN.